The MABEL Teleoperation Control Stack: A Complete and Honest Technical Reference

Frames, Retargeting, Whole-Body Control, Mobile-Base IK, the Tip-Over Motion Model, Self-Collision Avoidance, and Safety

MABEL Project — Control & Teleoperation

July 28, 2026

Abstract

This is the single, end-to-end reference for MABEL's teleoperation control stack: an operator wearing a head-mounted tracker (Apple Vision Pro, but the design is device-agnostic) drives a mobile bimanual humanoid — a 3-module swerve base, a cascaded vertical lift, a leaning torso, a 3-DoF neck, two 7-DoF arms, and two 17-DoF ORCA hands. We follow the data the whole way through, from raw operator poses to robot actuator targets, deriving every optimization, stating every control law, and — critically — being explicit about every *hardcoded* constant and every *design decision* taken along the way (the `--lock` mode, the collision-avoidance integration, the ORCA-wrist treatment, the objective-weight balancing). All robot-side quantities are computed by forward kinematics on the robot model (no privileged simulator state), so the same code runs in simulation and on hardware. The document is honest about the present state: two implementations coexist and are being consolidated (§1); we say which is which throughout.

Keywords: teleoperation, humanoid whole-body control, inverse kinematics, control barrier functions, swerve drive, tip-over stability, retargeting, MuJoCo.

Contents

1 State of the codebase (read this first) **2**

2 The pipeline at a glance **3**

3 Frames and the operator→robot map **3**

4 Hand retargeting optimization **4**

5 Arm IK optimization **5**

5.1 The ORCA wrist as a jointly-optimized DoF 6

6 Neck optimization **7**

7 Whole-body control **7**

8 Mobile-base inverse kinematics (swerve) **9**

9 Base motion model (tip-over safety) **9**

10 Self-collision avoidance **10**

11 Safety: fail-safe engagement and clutch **11**

12 Deployment: the bridge integration **12**
12.1 The direct joint-target route: measured semantics (2026-07) 13

13 Experiments **13**
13.1 End-to-end validation harness (2026-07) 14
13.2 Retargeting latency: where the operator's lag actually lives 15

14 The configuration registry: `control_param.json` **17**

15 Every hardcoded part, in one place (honesty section) **18**

16 Limitations **19**

1 State of the codebase (read this first)

Two control stacks currently exist and are being merged into one library:

- **Production controller** — the package `controller/mabel/`. It is installable (`pip install -e controller/`, or `controller/install.sh`; `import mabel`) and exposes a frontend-agnostic facade `MabelTeleop` mapping an `OperatorPose` to a `RobotCommand`. Since the 2026–07 reorganisation the package is structured by control responsibility: `sources/` (device ingest: wire / Vision Pro ZMQ / mock → `OperatorPose`); `hand_retargeting/` (keypoint IK, vector, anatomical — position-controlled, decoupled from the body); `whole_body_retargeting/` (clutch wrist SE(3) map, cosine arm IK, the decoupled neck look-at, lazy followers, per-operator calibration); `motion_model/` (the lift-aware tip-over/ZMP envelope `tip_over.py` — NumPy-only, re-exported by `config` — plus live CoM measurement and the calibrated base response); `dynamic_models/` (the MuJoCo dynamics oracle, the generalized-momentum observer, and the three control regions `arms/`, `upper_body/`, `whole_body/`); and `whole_body_controller/` (differential IK, the Day-1 PD position controller, gravity compensation, capsule collision avoidance, admittance = wrench estimation + base glide, region-selectable compliance, the swerve base controller, and the `WBCEngine` runtime). `plant.py` is the single sim/real seam. Every old flat module path (`mabel.wbc`, `mabel.swerve`, `mabel.dynamics`, ...) remains importable through compatibility shims.
- **Research / simulation stack** — the package `simulation/mabel_mujoco/scripts/teleop/`, driven by `avp_teleop.py`. It re-implements retargeting with a *cosine-similarity* formulation (the subject of most of §4–§7), and adds the safety layer (§11) and the arm self-collision projector (§10).

Where a component differs between the two, both are described and the difference is flagged. The *shared, single-source* pieces — the swerve IK (§8), the tip-over motion model (§9), and the capsule self-collision guard (§10) — live in `mabel` and are used by both. The safety primitives (§11) were developed in the research stack and have been ported into `mabel.safety`.

Single deployment surface. As of this revision the production WebSocket bridge (`server/sim_teleop_bridge.py`) drives the robot *exclusively* through `MabelTeleop`: every frontend

(Apple Vision Pro, iOS, web) sends the same `teleop_frame` wire payload, which the package itself ingests (`OperatorPose.from_teleop_frame`, `mabel.wire`), and the bridge issues one `teleop.update()` per physics substep. All control orchestration — mode/region switching, the whole-body torque engine, the direct joint override, the operator push, the clutch and the fail-safe — now lives behind the facade (§12), so the controller can be modified without touching the bridge or the apps.

2 The pipeline at a glance

The control loop is a feed-forward flow with two cross-cutting layers (safety and self-collision) wrapped around it:

$$\text{operator pose} \xrightarrow{\S 3} \text{frame map} \;\to\; \Big[\underbrace{\text{hand retarget}}_{\S 4},\; \underbrace{\text{arm IK}}_{\S 5},\; \underbrace{\text{neck}}_{\S 6}\Big]$$
$$\to \underbrace{\text{whole-body cascade}}_{\S 7} \to \underbrace{\text{base motion model}}_{\S 9} \to \underbrace{\text{swerve IK}}_{\S 8} \to \text{robot.}$$

The **self-collision** filter (§10) projects the arm targets onto a barrier-safe set; the **safety supervisor** (§11) eases the whole body to a home pose on tracking loss and re-engages on recovery. The robot is *physically simulated*: retargeted joint angles drive position-servo actuators with a feed-forward gravity torque $g(q)$ so the arms/lift/torso hold their pose without PD sag.

3 Frames and the operator→robot map

Operator poses arrive in an ARKit-style world: $+Y$ up, $+X$ right, $-Z$ forward. They are carried into the robot world (Z up, $-X$ front, $+Y$ right) by a fixed rotation A, anchored *once* at calibration by the pair $(p_0^{\text{op}}, p_0^{\text{rob}})$ = (operator head, robot head via FK):

$$W(p) = p_0^{\text{rob}} + s\,A\,(p - p_0^{\text{op}}), \qquad A = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad s = \text{REACH_SCALE}. \tag{1}$$

A maps forward→front, up→up, right→right. **[hardcoded]** A is a fixed permutation; in the production stack it is `config.AVP_TO_ROBOT` and the relative-rotation variant composes it with a fixed `WRIST_ALIGN` (identity unless a headset needs a calibrated wrist-axis correction). Because the anchor is a *world* reference, the robot tracks the operator's global translation *and* heading (walking, turning), not just head-relative motion.

Reach calibration (design decision). s is *not* a guess: the operator extends both arms straight forward; we measure the human reach (head→wrist, mean of the tracked hands) and the robot reach (a self-contained damped-least-squares extension of the arm), and set s = robot reach/human reach (clipped to $[0.5, 3.0]$, default 1.2). Only the *arm* targets are scaled by s; the finger optimization is fully decoupled with its own per-finger scales (§4).

Capture rule (redesigned 2026-07-19). The app-driven capture (`engine._calibrate_step`) locks the scale when *the arm-length number stabilises*: over a rolling 1.0 s window the reach samples must (i) number $n \geq 3$, (ii) span ≥ 0.5 s of wall time, and (iii) have standard deviation ≤ 2 cm; after 0.25 s of sustained stillness the window *median* is captured. The previous rule (≥ 5 samples

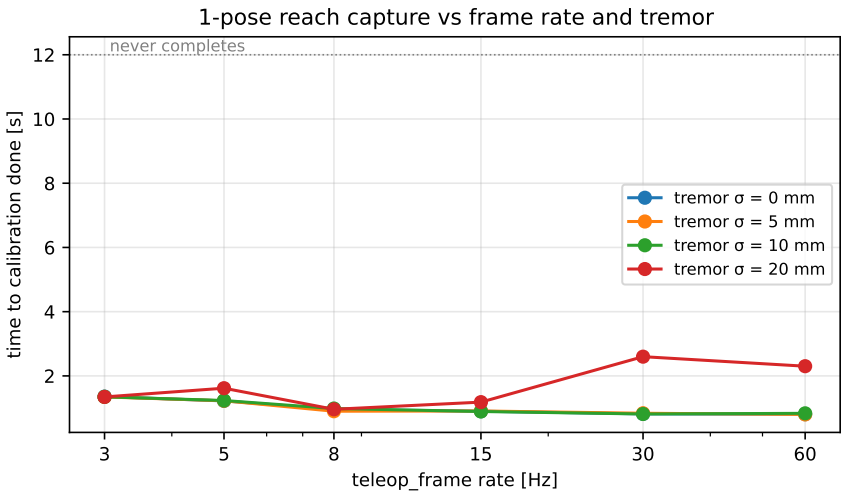


Figure 1: Reach-capture completion time vs. `teleop_frame` rate and synthetic wrist tremor σ under the redesigned stillness rule (span ≥ 0.5 s, $n \geq 3$, std ≤ 2 cm, 0.25 s hold, median capture), measured end-to-end over the WebSocket wire. Every cell completes; under the old count-and-range rule the ≤ 5 Hz column and the high-rate $\sigma = 20$ mm cells never did. Regenerate with `server/tools/exp_calibration_robustness.py` (seeded, 2 trials/cell).

inside a 0.6 s window with $\max - \min \leq 5$ cm) failed in both directions, measured over the real wire by `server/tools/exp_calibration_robustness.py` (Fig. 1): below ~ 8 Hz effective frame rate five samples never fit the window, so the capture could *never* complete (a loaded headset app streams that slowly), while under 2 cm operator tremor the range statistic grew with the sample count, so a *faster* stream was *more* likely to fail (60 Hz failed where 8 Hz passed). Time-span plus standard deviation are rate-independent, and the median rejects single ARKit outliers: after the redesign all 24 sweep cells (3–60 Hz $\times$ 0–20 mm tremor, 2 trials each) complete in 0.8–2.6 s; before it, 10 of 24 never completed. The Vision Pro app mirrors the rule on-screen (`CalibrationCoach`) and additionally displays the server's own live median (additive `calibrationStatus.reachLive/spread/hold` telemetry), so the ring can never fill ahead of the capture the server is actually timing.

4 Hand retargeting optimization

Goal: map the operator's finger skeleton to the 16 actuated finger DoFs/hand so the robot fingertips do what the human's do — including *touching* when the human's touch.

Canonical palm frame (key design decision). Each finger keypoint is matched in a *canonical palm frame* (forward = down the middle finger, side = index→pinky, normal = palm out) built *independently* from the operator anatomy and from the ORCA geometry. Because both frames come from their own hand, a given anatomical direction aligns *by construction*; this removed a hand-guessed rotational offset `R_offset` that the earlier version needed (and which made hand orientation track poorly). The frame is built from the named knuckle/metacarpal joints when present, else from the 21-point MediaPipe array.

Cosine-similarity objective (research stack). For the 16-DoF finger vector q, with robot keypoint set $\{r_k(q)\}$ (FK in the canonical frame) and scaled human keypoints $\{h_k\}$, minimize

$$\mathcal{L}_{\text{hand}}(q) = \underbrace{W_c\Big(1 - \tfrac{\langle r,h\rangle}{\|r\|\,\|h\|}\Big)}_{\text{direction}} + \underbrace{W_r \sum_{(a,b)} \big\|(r_b - r_a) - (h_b - h_a)\big\|^2}_{\text{relative (pinch) distances}} + W_s\|q - q_{\text{prev}}\|^2 + W_{\text{reg}}\|q\|^2, \qquad (2)$$

where r, h are the stacked keypoint vectors. The relative-distance term over the thumb↔fingertip pairs is what makes the robot fingertips meet when the operator pinches: two coincident operator tips map to one robot point regardless of scale, so the gap goes $0 \to 0$. Solved with SLSQP. **[hardcoded]** the per-finger scales `SCALES` (default all 1.0), the weights W_c=20, W_r=150, W_s=0.5, W_{reg}=0.02, and the finger/joint name maps.

Production stack alternative. `HandIKRetargeter` solves the same correspondence by Gauss–Newton (4 iterations, $\approx$ 0.5 ms/hand, warm-started, background-threaded at 20 Hz) — much faster than the SLSQP reference ($\approx$ 3 ms). A legacy frame-invariant angle-extraction mapping (`hand_retarget`) and a pinch-only fallback (`hands.apply_grasp`) also exist.

Live app path (since 2026-07). On the WebSocket app path (Vision Pro / iOS / web via `MabelTeleop` → `TeleopEngine`) the fingers run `scripts/teleop/hand_anatomical.py`: a validated line-faithful port of the control-center GUI's *webcam* anatomical decomposition (`bodyteleop.decomposeHand` + the calibrated ORCA ranges, PIP+$\frac{1}{2}$DIP coupling, thumb opposition from the metacarpal's dive below the palm plane), so every input device — webcam or headset — maps hands through one algorithm. Parity with the actual JS is regression-tested to machine precision (`scripts/tools/exp_vp_app_teleop.py`, E1: $\max\Delta = 6.7\times10^{-16}$ rad over 200 random hands; note the chirality sign flips for true-ARKit skeletons because MediaPipe labels handedness on the selfie-mirrored convention). The SLSQP optimizer (2) remains the desktop-entry solver and still provides the canonical robot palm frame to the arm retargeter. The same 2026-07 rewiring restored the arm cost's elbow term on the headset path (elbows derived from ARKit `forearm_arm`) and the base yaw-follow (elbow-line `body_yaw` estimated in-engine); see `teleop_system.pdf` §6.

5 Arm IK optimization

Goal: place each hand at the operator's wrist/contact target with a *human-like* posture — no weird twist, graceful near singularities, fast.

Why cosine SLSQP, not Jacobian DLS. The research arm IK lifts the finger optimizer's idea to the arm: it matches *directions* and a contact point rather than inverting a Jacobian, so it is smooth and never erratic at singularities (no J^{-1}). The optimized vector is $q = [\text{Arm}_1 \ldots \text{Arm}_7, \text{wrist}]$ (§5.1). The cost is

$$\begin{aligned}
\mathcal{L}_{\text{arm}}(q) = {} & W_{\text{wrist}}\,\|c(q) - c^\star\|^2 && \text{(contact position — primary)}\\
& + W_{\text{elbow}}\big(1 - \hat{e}(q)\cdot\hat{e}^\star\big) && \text{(elbow direction } \textit{from the shoulder}\text{)}\\
& + W_{\text{orient}}\big[(1 - f\cdot f^\star) + (1 - u\cdot u^\star)\big] && \text{(palm forward + normal axes)}\\
& + W_{\text{smooth}}\|q - q_{\text{prev}}\|^2 + \sum_i \rho_i\,(q_i - q_i^{\text{nom}})^2. && \text{(temporal + per-joint posture)}
\end{aligned} \qquad (3)$$

The contact point $c(q) = \text{wrist}(q) + \text{HAND_REACH} \cdot R_p(q)\,\hat{x}$ sits a fixed distance ahead of the ORCA wrist along the palm, so bimanual fingertip contact maps to one robot point (scale-invariant, like the fingers). The elbow term takes the direction *from the shoulder* (not the head) — this is the single most important anti-twist term: it pins the redundant elbow to the human's, stopping the arm from "flipping" into back-of-robot configurations.

Objective-weight balancing (experimentally tuned, §13). The balance principle, confirmed by a sweep: position must dominate (W_{wrist}=600 $\gg$ the rest) or the hand misses; the elbow term shapes the null-space but if it climbs ($\gtrsim 100$) it *fights* the wrist and tracking degrades — W_{elbow}=30 is the balance; orientation W_{orient}=20; smoothness W_{smooth}=2 is the sweet spot (1 jitters, ≥ 4 lags and loses accuracy). The per-joint posture weights ρ_i are a *mobility prior*: **[hardcoded]** `ARM_REG_W`= $[0.08, 0.08, \mathbf{0.40}, 0.08, \mathbf{0.02}, 0.08, \mathbf{0.60}]$ — hold shoulder-roll (Arm3) hard so the roll is done by the forearm, leave forearm-roll (Arm5) free so the hand can flip up, and hold wrist-yaw (Arm7) hard.

Wrist-yaw constraint (design decision). A human wrist barely yaws (radial/ ulnar deviation $\approx \pm 25°$). So Arm7's optimization bound is *hard-clipped* to rest $\pm$ARM_J7_YAW_LIM $= 0.45$ rad (intersected with the model range), forcing any palm flip onto the forearm and the ORCA wrist rather than letting the solver spin the last joint to fake orientation. This was added after observing exactly that failure mode (Arm7 spinning, arm twisting).

The `--lock` mode (design decision). A CLI flag fixes Arm7 at its rest value, giving a 6-DoF arm + the ORCA wrist. It was added for hardware where Arm7 is unreliable or for a simpler kinematic chain; the IK then optimizes 6 (or 7 with the wrist) joints and Arm7 is held at `j7_lock_val`.

5.1 The ORCA wrist as a jointly-optimized DoF

This was a deliberate change with a measured justification. The ORCA hand has a 1-DoF wrist (pitch). Originally it was *hardcoded*: the operator's 1-DoF wrist flexion was projected onto it (`set_wrist_flex`), decoupled from the arm IK. We made it an *8th jointly-optimized DoF* (solved from the orientation objective, `ARM_OPTIMIZE_WRIST`=True, light regularization `ARM_REG_WRIST`= 0.03). Rationale: the extra DoF adds redundancy that the orientation term can use to flip the palm up/down *without* twisting Arm7 or the shoulder. Measured head-to-head (§13), even giving the hard-coded mode the oracle (ground-truth) wrist, the jointly-optimized wrist tracked better (contact 23 → 20 mm), matched orientation better, and twisted less (wrist-yaw 9.1 → 7.1°). End-to-end IK accuracy improved 30 → 16 mm (median). The +2 ms cost of the 8th DoF was recovered by lowering the SLSQP iteration cap 45 → 35 (the wrist DoF aids convergence). When hardcoded mode is selected, `set_wrist_flex` maps the human flexion with **[hardcoded]** `WRIST_FLEX_GAIN`= 1, `WRIST_FLEX_SIGN`= 1.

Production stack alternative. `WholeBodyIK` tracks the palm SE(3) pose with weighted damped-least-squares: $\dot{q} = W^{-1}J^\top(JW^{-1}J^\top + \lambda^2 I)^{-1}v_{\text{des}}$ plus a null-space posture pull to q_{rest}, integrating lift + torso + 7 arm + wrist per side (8 DoF/side). It is faster per step but can be erratic near singularities; the cosine method trades a little speed for robustness.

6 Neck optimization

The research stack maps the operator's *relative* head rotation (vs. calibration) into the 3-DoF neck: $\Delta R = A\,(R_h R_{h,0}^{\top})\,A^{\top}$ gives yaw/pitch/roll, each scaled by `NECK_GAIN`= 1 and a per-axis sign ([hardcoded] yaw +, pitch −, roll + — the pitch sign was flipped after the head tilted the wrong way). The production `NeckController` instead solves an active-perception *look-at-point*: given a gaze target it computes the 3-DoF neck that points `HEAD_FORWARD_LOCAL` at the point (HoMMI-style), used in both teleop modes so the robot looks where the operator looks.

7 Whole-body control

Goal: coordinate the redundant body DoFs (torso, lift, base) that *extend* the operator's reach, so the arms stay inside their comfortable workspace and the wrist never ends up behind the body.

Head-displacement bounding box (research stack). The base-engage criterion is an asymmetric, front-of-head box on the operator's head displacement $\Delta = A(p_{\text{head}} - p_{\text{head},0})$, decomposed into forward $f = -\Delta_x$ (robot front = $-x$) and lateral $\ell = \Delta_y$. The part *inside* the box, $f_{\text{in}} = \text{clip}(f, -b_{\text{back}}, b_{\text{fwd}})$, is absorbed by the **torso** (it bends $\propto f_{\text{in}}$, the fast first responder); only the *overflow* $f - f_{\text{in}}$ drives the **base**, with a position target $\propto w(f - f_{\text{in}})$ resolved by a P-controller to a body velocity. The box is [hardcoded] deep in front ($b_{\text{fwd}} = 0.18\,\text{m}$, so leaning/reaching forward uses torso+lift, not the wheels) and shallow behind ($b_{\text{back}} = 0.02\,\text{m}$, so backing up engages the base *immediately*); the per-direction weights ($w_{\text{back}} > w_{\text{fwd}}$) and the high gain $K_p = 3.5$ make backing up the most responsive. These are pure sensitivity *gains*, not separate objectives. The **lift** follows head height (speed-limited), and the operator's body heading drives the base yaw.

Front-line clamp (design decision — "no twist to the back"). Even an eager base cannot out-run a fast operator, so we additionally *hard-clamp* the wrist/contact target to stay in front of the body's vertical line: in the chassis frame, $x_{\text{contact}} \le \text{ARM_WRIST_FRONT} = -0.05\,\text{m}$. The arm therefore *physically cannot* be stretched/twisted behind the robot, regardless of base lag; the base then backs up to bring the operator's intent forward of the line. Because the line rides with the chassis, the system is stable (the base moving back releases the clamp).

Production stack alternative. `lazy_base.LazyBase/LazyLift/LazyTorso` are reactive bounding-box followers keyed to the *hand* targets (rather than the head), with their own `LAZY_*` envelopes; same intent (keep the hands reachable), different signal.

Task-space (wrist-method) whole-body reach (2026-07). The wrist method's dragged/streamed palm targets now get the same reach cascade, via three additions (measured end-to-end in the Control Lab; full experiments and the defect chain in `web_gui/control_lab/docs/control_lab.tex` §Wrist task-space reach):

- **Sync-Lift units in `WholeBodyIK`.** The lift is a tendon-cascaded stage pair (`Sync Lift`: Upper = Lower; ctrl = stage *sum*). The IK now keeps `target_q["Lift"]` in joint units, doubles the lift Jacobian column, clips to the stage joint range, writes ctrl = 2×target, and moves *both* stage joints in every scratch-pose FK — dragging the targets up recruits the lift at the 60 mm/s hardware ceiling (0.442 m in 8 s, palm settling at 22.6 mm). Consumers mirror the target onto the wire as `lift_lower=lift_upper=target`.

- `task_space.BaseFollow` — **the base is the EAGER reach DoF.** The base is not an IK DoF; this follower (new module in `whole_body_controller/`) is a *hysteretic position+heading servo* implementing the human-like priority (operator directive): the chassis drives to its HOME-pose offset from the target centroid (the working distance the arms were designed for, ∼80% of the measured 0.67 m usable reach — ∼2 cm of drag engages) and the heading squares the shoulders to the target PAIR axis $p_r - p_l$ (wrapped ±90°): equal-lateral drags STRAFE (a centroid-bearing turn left the far palm 200 mm short), a rotated pair YAWS the robot with the operator's body turn; the centroid bearing is only the degenerate-pair fallback and the come-about override, min-distance-gated (the ungated bearing servo measured a −3.3 rad/s spin limit cycle on arrival). Both servos are snap-stop hysteretic (engage > `_ON`, decisive minimum speed, ZERO below `_OFF` — a plain P-law floats near its deadband), all errors are planar, the LAZY lift/torso (weights 50/30) are eased to rest while `.active` ("don't lean into what you're walking to"), a deliberate stick wins, and everything routes through `safe_base_step`. Edge-case sweep (12 cases: fwd/back/left/right/up/down/ squeeze/spread/diagonal/rotate, realtime): all settle 0.8–5.2 s, float ≤1 mm std, zero residual twist, pitch ≤0.5°.
- **Static-stability: rate caps + a CoM net.** Chasing an out-of-reach target, the DLS slams the torso and the swing's angular momentum tips the robot (MABEL's home support margin is only 0.094 m; full both-arm forward reach is statically infeasible). `MAX_TORSO_IK_SPEED` (= 0.5 rad/s) removes the slam — natural trajectories then bottom out at +0.014–0.031 m of live margin without falling — and `WholeBodyIK.apply` carries a last-ditch CoM guard *below* that band (`COM_GUARD_MARGIN`= 0.010 m): near the line it projects $\dot{q}$ onto the margin-preserving tangent (CBF damper on the finite-differenced support margin). A uniform stop-scale ratchets into folded-arm attractors (0.6–0.9 m of lost reach), which is why the net does not shape normal operation.

The same audit fixed two latent lift-unit bugs on the hold path: `TeleopEngine.hold_current` now adopts the measured lift extension (the tendon actuator is excluded from the joint-transmission hold list, so a Stiff re-entry used to replay a stale ctrl), and the facade's `_measured_ctrl` no longer indexes `jnt_qposadr` with the Lift's *tendon* id (torque re-enable reseeded a stored lift target from the free joint's x).

Torque engine — Hold (impedance) vs Soft (compliance), by region. `WBCEngine` (the *same* object the standalone `scripts/compliance.py` drives) runs the `joint`/`wrist` methods as a per-joint torque law $\tau_j = b_j(q,\dot{q}) + K\,(q_j^{\mathrm{des}} - q_j) - D\,\dot{q}_j$ on the managed joints (their stiff position servos are neutralised on `enter`, so they are torque-driven and back-drivable). **Hold** = impedance ($K > 0$, the app's stiffness slider): it springs to the set-point q^{des}, which the app supplies via joint targets. **Soft** = compliance ($K \equiv 0$): pure gravity/bias cancellation + viscous damping, so the arm floats and is dragged freely, and $q_j^{\mathrm{des}} \leftarrow q_j$ is *re-anchored to the live pose every tick* — the operator drags the robot (in the Mac sim, the desktop viewer, or by hand on the real arm) and it *holds where left*, with a seamless hand-off to Hold (the set-point already equals the dragged pose). In Soft the engine takes *no* position target from the app on the managed arm/torso joints (only grips/lift/ fingers still track); the small residual sag is the gravity-comp model error, not a spring. Three **regions** parameterise how much of the body yields, exactly mirroring the app's picker: `arm` (arms + wrists), `upper_body` (+ torso + lift, both gravity-comp), and `whole_body` (+ the mobile base *glides* under the net push: `external_base_wrench` reduces the arm-body wrenches to a planar (f_x, f_y, m_z), `BaseAdmittance` integrates it, and §9's tip-safe ramp drives the swerve). The operator push `xfrc_applied` (app `external_force` *plus* the desktop-viewer Ctrl+drag perturbation)

is composed *once per substep, before* the control law, so the whole-body base-wrench estimate sees the same forces `mj_step` applies.

Reference smoothing (design decision — "track the wrist slower but surely"). In Hold, the impedance set-point q^{des} for the `wrist` method arrives over the wire as a *multi-joint* IK solution (the green-ball planner, a wrist drag, or a streamed Cartesian goal). Its null-space / redundancy resolution can flip between wire frames, so the raw q^{des} *jumps* at the 60 Hz command rate. Fed verbatim into the spring $K(q_j^{\text{des}} - q_j)$ with the modest joint damping $D = 0.6$, every jump is a torque impulse and the arm *buzzes* — the operator's report that "task-space wrist tracking is jittery, unlike the smooth joint slider" (the slider is smooth only because *its* q^{des} moves slowly and one joint at a time). We therefore filter the *reference* the spring tracks, at the single choke point in `WBCEngine.step`: a dt-correct first-order low-pass (time constant $\tau_{\text{ref}} = 0.08\,\text{s}$) rejects the high-frequency IK wiggle, and a slew cap ($\dot{q}_{\text{ref}} \le 3.0\,\text{rad/s}$) turns a large jump into a deliberate ramp rather than an impulse. A *smooth, slow* command — the joint slider, or the client-eased green-ball Execute (joint-space smoothstep) — sits below both the corner and the cap, so it passes with only a few tens of ms of lag; the reference is seeded to the measured pose so the Soft→Hold hand-off stays bumpless. Constants are `WBC_REF_TAU` / `WBC_REF_SLEW` (env-overridable); the fix is server-side, so it lands for iOS, Vision Pro, and web at once. Quantified in §13.1.

Sticky control spec (design decision — "Soft must not snap back"). The bridge follows one active driver, but the *spec* (method/type/region) is now *sticky*: it changes only when the active client sends an explicit `control_mode`, and otherwise persists. Previously, once an operator engaged Soft and then dragged the robot without sending app input, the driver claim expired after `CMD_HOLD_S`= 4 s (idle frames carry no claim) — or a brief WebSocket reconnect handed the client a fresh id with no spec — and the effective spec reverted to the navigation default, which `exited` the torque engine and snapped the arm back onto its position servos. Holding the last explicit spec keeps the robot gravity-compensated until a client *deliberately* changes mode (leaving the Body tab sends navigation), which is also the safe idle default for a home robot.

8 Mobile-base inverse kinematics (swerve)

`swerve_ik(`v_x, v_y, ω`,data)` turns a body-frame twist into the three modules' steering angles and wheel speeds, $v_{\text{module}} = v_{\text{com}} + \omega \times r_{\text{module}}$. Two practical refinements: *shortest-path steering* (never sweep a steering column $> 90°$ — reverse the wheel instead), and an *anti-slip cosine choke* drive $\times = \cos(\text{steer error})$ when the steering error exceeds **[hardcoded]** a 0.52 rad deadband, so wheels don't scrub while reorienting. This is single-source in `mabel.swerve` and shared by both stacks.

9 Base motion model (tip-over safety)

Goal: hand a desired base velocity to the wheels without tipping the robot, especially with the lift raised.

The realized velocity is ramped toward the target through a dynamic envelope whose speed/accel caps shrink as the CoM rises. Braking at $|a|$ tips the robot over the support edge it pushes against (lever d, CoM height h): $mah \le mgd \Rightarrow a_{\text{tip}} = g\,d/h$. Raising the lift raises $h \approx z_0 + kz$, so the safe accel and speed scale as $\sim 1/h$. The binding direction is the *front* edge (MABEL's triangular base has a single rear wheel and a slightly forward CoM), so forward braking is kept

conservative (safety factor 2.0) while lateral/rear run near the tip limit (SF 1.2) for snappy reversals. For combined motion we enforce the full support-polygon (ZMP) criterion — the zero-moment point $x_{\text{zmp}} = x_{\text{com}} - (h/g)a_c$ must stay inside the wheel-contact triangle, with a_c including Coriolis/centripetal/ tangential terms. `config.safe_base_step` does one tip-safe ramp step; it is the *single source* shared by the teleop smoother and the stability tests. **[hardcoded]** the levers and CoM fit (re-derived 2026-06 after the 0.10 m lift-base drop, see the stability study: `TIP_LEVER_FWD`= 0.078, `TIP_LEVER_BACK`= 0.241, `TIP_LEVER_LAT`= 0.108, `COM_Z0`= 0.369, `COM_DZDLIFT`= 0.451) are *measured from the chassis support polygon in the XML*; if the chassis or payload changes they must be re-measured (`experiments/wbc_stability.py` E4 flags drift; currently 0.2 cm XY bias, 1.5 cm height-fit error).

Validated envelope (measured). `experiments/exp_wbc_validation.py` (V1–V2) verifies the model: the per-axis speed/decel caps are monotone non-increasing in lift height (forward speed $0.80 \to 0.47$ m/s, forward decel $1.04 \to 0.58$ m/s^2 over the lift range); per-axis steps and reversals never exceed the per-axis accel caps and hold ZMP usage ≤ 1; a stop command from an over-limit twist at full lift converges in 0.74 s (no deadlock). One honest caveat the tests surfaced: usage is *quadratic* in the twist, so the path between two individually admissible *combined* twists (drive + spin) is not necessarily admissible — during a target switch the instantaneous usage can transiently exceed 1 (max 1.07 within the teleop-reachable command envelope $|v| \leq 0.8$, $|\omega| \leq 1.5$; max 1.51 under an adversarial unrestricted sweep). The accel clamp's contract $u \leq \max(1, u_0)$ held in all 450k random steps, and physically replaying the adversarial worst case on the floating-base model peaks at 4.2° chassis tilt (the same commands raw fall at $>80°$); the transient is bounded by the ≥ 1.2 safety factors and is a documented property, not a hazard.

Reverse guard (design decision, from an experiment). A simulation sweep found that at full lift the robot tips when *reversing*: accelerating backward throws the CoM over the short front lever, and the rigid ZMP model under-protected the compliant articulated robot. We added a reference-governor *reverse guard* that scales backward velocity to zero as the lift rises (frac $= 1 - z/z_{\max}$); forward/ lateral/turning pass through. With it, every reachable command stays upright across the full lift range (§13). It is a swappable `base_filter` on the whole-body controller — the same hook intended for a future control-barrier/geofence.

10 Self-collision avoidance

Goal: a hand can never be driven into the torso, head, or the other arm.

Capsule guard (single source, `mabel.self_collision`). The two forearms and hands (which move under the arm DoFs) and the torso + head (which they crash into) are wrapped in capsules; for a pair the surface gap is $d(q) = \|c_a - c_b\| - r_a - r_b$ with c_a, c_b the closest points on the core segments (Ericson's segment–segment test, so no dependence on a particular MuJoCo version). The discrete control-barrier constraint on the gap rate $\dot{d} = n^\top (J_a - J_b)\dot{q}$ is

$$-\,n^\top (J_a - J_b)\,\dot{q} \;\leq\; \alpha_c\,(d - d_s) \qquad \text{for } d < d_{\text{inf}}, \tag{4}$$

i.e. the gap may shrink ever more slowly as it nears the safety distance d_s. The guard offers three laws: a uniform velocity *scale* ("stop"), a tracking *QP*, and a torque *barrier force*. **[hardcoded]** the radii (`SC_RADII`: forearm 0.035, hand 0.055, torso 0.09, head 0.12 m) and the monitored pair list.

Arm projector (research stack, design decision). The cosine arm IK can drive a hand into the body, so we keep the natural IK pose q_{des} and *project* it onto the safe set with a one-step velocity QP over *both* arms jointly:

$$\min_{\dot{q}} \tfrac{1}{2}\|\dot{q} - \dot{q}_{\text{des}}\|^2 \quad \text{s.t.} \quad G\dot{q} \le h,\ \dot{q}_{\text{lo}} \le \dot{q} \le \dot{q}_{\text{hi}}, \qquad q_{\text{safe}} = q_{\text{cur}} + \dot{q}\,\Delta t, \tag{5}$$

with G, h assembled from (4) and $\dot{q}_{\text{des}} = (q_{\text{des}} - q_{\text{cur}})/\Delta t$ at the frame interval (so α_c [1/s] is dimensionally correct). The hand thus *slides* along the obstacle while tracking as closely as the barrier allows. **It is ON by default** everywhere as of 2026-07 (`config.SELF_COLLISION_ENABLE=True` seeds `WholeBodyIK`, `WBCEngine` *and* the `TeleopEngine` projector; the bridge exposes an explicit opt-out, `--no-self-collision` / `MABEL_SELF_COLLISION=0`, and the standalone teleop `--no-collision`) — previously the guard was opt-in and three of the four entry points started unguarded. A related bug was fixed at the same time: the engine gated the projector on "was ever built" instead of the enable flag, so `set_self_collision(False)` did not actually disable a once-enabled guard. When on, it is a strict no-op (targets pass through, no QP, no velocity clamp) until a monitored gap drops below the influence distance — so it never slows free motion. We use a slightly more conservative influence/gain (`SELFCOL_INFLUENCE`= 0.10, `SELFCOL_ALPHA`= 8) than the shared defaults so the margin holds under the snappier (low-smoothing) arm. Measured: a hand marched into the chest holds +4.1 cm (margin 2.5 cm) where the unguarded arm penetrated to −3.5 cm; the QP costs ≈ 0.4 ms.

Why the software guard is the *only* arm–arm defense (measured). The MJCF deliberately disables arm–arm contact pairs (the arm/hand collision geoms carry `conaffinity=0`, so two `contype=1`/`conaffinity=0` geoms never generate a contact): with the guard off, crossing the arms passes the hands *through* each other with no contact force at all. Hand–torso pairs do contact physically, but only after the meshes touch — well inside the capsule margin. The end-to-end validation (`exp_wbc_validation.py` V3, §13) therefore treats the capsule guard as a hard requirement, and the default-ON change above is load-bearing for safety.

11 Safety: fail-safe engagement and clutch

Tracking-loss fail-safe (`EngagementSupervisor`). It watches the freshness of the stream, head, hands, and elbows and drives one rate-limited blend scalar $\alpha \in [0, 1]$ applied to the upper-body position actuators + the lift: ctrl = $(1-\alpha)$ home + α live. On loss (headset off → stream stalls, or a limb drops out) α ramps to 0 over `SAFE_RETREAT_TIME`= 4 s, easing the upper body to a safe HOME pose (rest arms, **[hardcoded]** lift lowered to 0) and freezing the base; on re-acquire α ramps to 1 over `SAFE_ENGAGE_TIME`= 2.5 s with the live target being the operator's *current* pose, and the world map (1) is re-anchored on the rising edge so there is no jump. **[hardcoded]** the timeouts (0.4 s stream, 0.5 s signal). A `stop` console command parks the robot at HOME the same way. *Signal-adaptive gating:* the elbow term is *optional* — the desktop ZMQ stream and (since 2026-07) the webcam web client carry elbows, and the headset wire path now derives them from ARKit's `forearm_arm` joint (the elbow end of the forearm), so all paths feed the arm cost's W_{elbow} term; the supervisor requires the elbow's freshness only once an elbow has actually appeared. (The wire accepts explicit `leftElbow`/`rightElbow` transforms, which win over the derived ones; a present elbow is mapped through the same world map as the contact point — see the webcam retargeting doc, `simulation_control_center/docs/retargeting.pdf` §5.3.) Without this, a head+hands frontend could never satisfy the "complete frame" test and the robot would stay pinned at HOME. By the same logic the *hands* term is dropped in NAVIGATION mode (`require_hands`, cleared by the

facade when the active method is navigation): driving streams head + joystick only, so requiring hands would disengage the fail-safe and pin the lift at HOME (the base still drives, as the swerve actuators are not supervisor-managed) — the exact symptom of the right-stick lift going dead. Stream and head freshness still gate engagement, so a real link or headset drop still fail-safes while driving.

Clutch / re-indexing (`ClutchController`). A single-hand *double pinch* (either hand; hysteresis on each hand's thumb–index gap, two clicks within `CLUTCH_DOUBLE_WINDOW`= 0.6 s, a cooldown so one gesture is one toggle) toggles a whole-body freeze: the robot holds its pose and the base stops while the operator recenters; another double-pinch releases and re-anchors so the robot eases from its held pose to the operator's new pose. A single pinch is left free for grasping. **[hardcoded]** the pinch thresholds (0.03 on, 0.055 m off). *Design history:* this began as a two-handed sustained fist (too twitchy / collided with bimanual grasps), then a keyboard button, and settled on the either-hand double-pinch.

12 Deployment: the bridge integration

Every client — Apple Vision Pro, iOS, web — talks to one WebSocket bridge (`server/sim_teleop_bridge.py`) that owns transport only (client arbitration, cameras, recording, naming, security) and delegates *all* control to a single `MabelTeleop` instance. The apps are never modified to change the controller; the wire contract they already speak is sufficient.

Wire → pose. An app `teleop_frame` carries a head transform and, per hand, an `anchorTransform` plus a list of named joint `localTransforms` (ARKit frame). `mabel.wire` turns this raw payload into an `OperatorPose`: a joint's world transform is anchor · local; it builds the wrist/thumb-tip/index-tip list the retargeter reads, keeps the full named skeleton for the finger IK, and derives the thumb–index pinch gap that drives the clutch. The navigation stick is mapped to the controller frame by `app_joystick` (the forward/strafe axes are negated; yaw/lift pass through), so the body-frame convention is a one-line library edit rather than a per-app change.

Message → method. Each wire message maps onto one facade call: `control_mode`→`set_control_spec` (selects navigation / retarget / joint / wrist and, for the torque methods, (re)enters `WBCEngine` with the chosen type+region on the control thread); `joint_command`→`set_joint_targets` (direct overrides applied *after* IK so an explicit slider wins); `external_force`→`apply_external_force` (the world-frame wrench the compliance/whole-body engine admits); `reset`→`reset`; `recalibrate`→`recalibrate`. The bridge follows a *single active driver* so a second idle client can neither move the base nor change the mode.

One call per substep. The bridge owns physics: it accumulates real time and, per `mj_step`, calls `cmd = teleop.update(pose, joystick, dt)`. That one call runs the whole stack — clutch → (retarget + whole-body IK *or* the torque engine) → fail-safe → direct joint override — and returns a `RobotCommand` (`ctrl`, `engaged`, `clutched`, `mode`); on hardware the same call runs against a kinematic mirror and the `RobotCommand` is forwarded to the robot.

12.1 The direct joint-target route: measured semantics (2026-07)

The `joint_command` route (`_apply_joint_targets`) became the execution backend of the studio's choreography composer (`simulation_control_center/docs/choreography.pdf`), which exercised it far harder than sliders ever did. Its contract, established empirically against the live sim:

- **Coverage.** Both 7-DoF arms, both ORCA wrists, individual finger joints (`{side}_{finger}_{mcp,pip,dip,abd}`), `torso`, the lift (`lift_lower`+`lift_upper` are *summed* into the single Lift actuator), and a per-hand scalar `{side}_grip`$\in [0,1]$ that maps through the grasp table. **The neck has no route** — it is owned exclusively by the head-tracking path, a real limitation for scripted expressive motion.
- **Latched targets, partial commands.** Targets are held after the sender goes silent (verified: a raised-arm pose held for 8+ s with zero re-sends and no reclaim by idle clients). Joints omitted from a `joint_command` keep their previous targets — but a first command that names only one joint leaves every other joint on the controller's own output, so scripted motion should always send full upper-body poses.
- **Tracking.** Steady-state error < 0.05 rad in the workspace interior, up to 0.15 rad at gravity-loaded extremes (shoulder pitch commanded -1.20, read -1.05); held-pose gravity sag 0.07 rad to 0.14 rad (T-pose abduction 0.104, bow 0.135).
- **Arbitration.** A stick deflection > 0.04 or any `joint_command` takes the strong (active-driver) claim; idle pose-streaming clients observe without reclaiming.
- **Base interaction.** With no nav stream, arm reaction forces walk the free swerve base (0.221 m during one wave emote); streaming `navJoystick` zeros alongside pins it to 0.002 m. Drive gains measured through this route: $v = 0.80$ ly m/s (forward/reverse symmetric), $\omega = 1.50$ rx rad/s; open-loop turns under-rotate as angle grows (-1.6% at 90°, -11% at 180°, worse in clutter).
- **Release must zero (2026-07).** A *headless* driver (the web Control Center / a phone in base-driving streams a stick but no head pose) sets the base velocity target only from the stick. The unified `_apply_stick` path updated the target *only while the stick was deflected*, so on centering it *latched* the last non-zero twist and the base drove on non-stop — linear *and* yaw. Because a head-less frame has no head-follow term to overwrite the target, the centered-stick branch must now command $(v, \omega) = 0$ explicitly (a head-tracked driver still keeps its head-follow base target, so immersive teleop is unchanged). Verified across forward/strafe/yaw: driven $|v| = 0.80/0.80/1.50 \rightarrow$ released $|v| < 0.02$ (`simulation_control_center/tests/test_nav_joystick.py`). The lift is a rate jog, so it already stops the instant the stick releases.

13 Experiments

All numbers are regenerated headless by the harnesses `tools/teleop_benchmark.py` (the live pipeline on a scripted operator) and `tools/tune_retargeting.py` (the weight sweep), both under `simulation/mabel_mujoco/scripts/`. Fig. 2 collects per-stage latency, the tip-safe base envelope, and the self-collision barrier.

- **Speed.** Control/physics $\approx$ 500 Hz (0.6 ms/step, lidar sensors disabled). Hand retarget $\approx$ 2.7 ms to 3.7 ms, arm IK $\approx$ 5 ms (8-DoF, cold-start), self-collision QP $\approx$ 0.4 ms. Both retargeters run twice per VR frame ($\approx$ 120 Hz aggregate).

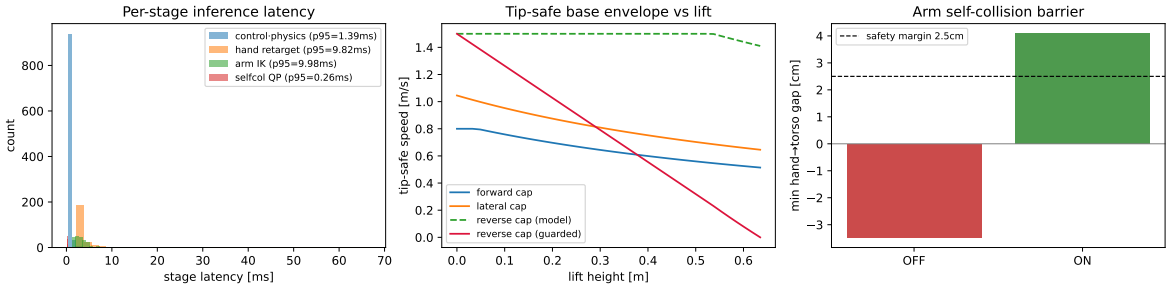


Figure 2: Left: per-stage inference latency. Middle: tip-safe base speed envelope vs. lift height (forward, lateral, model reverse cap, and the guarded reverse cap that goes to zero raised). Right: minimum hand→torso gap with the arm self-collision barrier OFF vs. ON against the safety margin.

- **Arm-IK accuracy.** On reachable targets (cold-start): 16 mm (p50)/ 65 mm (p95), improved from 30 mm p50 by the wrist DoF.
- **ORCA wrist: hardcoded (oracle) vs jointly-optimized.** contact 23→20 mm, orientation err 0.017→0.014, wrist-yaw twist 9.1→7.1°, natural-elbow fraction 23→60%. Optimizing wins on every axis.
- **Tip-over.** Across the reachable command envelope and lifts $\{0, 0.32, 0.635\}$ m, max chassis tilt $< 4°$; the reverse guard takes backward speed 0.8→0 m/s as the lift rises — all upright, where the unguarded model tipped.
- **Fail-safe / clutch.** Eases to HOME on loss (4 s) and back on re-acquire (2.5 s), re-anchored, no jump; the clutch toggles exactly once per double-pinch and ignores single pinches.
- **Weight sweep.** The current arm weights sit at/near the optimum; the only robust change was lowering `ARM_MAXITER` 45→35 (the wrist DoF aids convergence), and confirming W_{smooth}=2 as the jitter/lag sweet spot.
- **Direct joint route, live (2026-07).** The choreography composer's validation campaign (30 scripted actions, wire replays on the live sim) doubles as an end-to-end test of §12.1: every animation returned to neutral within 0.05 rad on all 14 arm joints, a 29.5 s five-action choreography executed cleanly, and a full 360° scripted pirouette closed within 2.4°. Methods, calibration tables and figures: `simulation_control_center/docs/choreography.pdf`.

13.1 End-to-end validation harness (2026-07)

`controller/experiments/exp_wbc_validation.py` exercises the FULL deployed stack — the `MabelTeleop` facade (SLSQP hand + arm retargeting, whole-body cascade, fail-safe, clutch, self-collision projector, WBC torque engine) — offline against real physics (`mabel_home.xml`, `mj_step`), with a fixed seed and a patched deterministic clock. Synthetic `OperatorPose` streams at 100 Hz drive four stressor trajectories; every run is repeated with the self-collision guard ON (default) and OFF. Table 1 gives the measured numbers (tracking error = distance from the robot contact point to the *commanded, front-clamped* target while it moves, so it includes trajectory-following lag).

WBC torque engine (V4). Joint-impedance steps on Right Arm 4 (+0.4 rad, damping fixed at D=0.6): rise time 0.16/0.09/0.06 s and overshoot 24/38/45 % at K = 20/50/100 Nm/rad —

	guard ON (default)		guard OFF			
trajectory	err m/p95 [mm]	gap [cm]	err m [mm]	gap [cm]	limit viol.	drift
cross-body reach	154 / 279	+5.4	70	+1.2	0 / 0	≤0.1 mm
overhead reach	42 / 94	+9.0	33	+8.8	0 / 0	≤0.1 mm
cross-arm swap	138 / 293	+4.4	111	**−5.7**	0 / 0	≤0.1 mm
chest poke (deliberate)	205 / 388	+2.5	58	**−2.5**	0 / **307**	≤0.1 mm

Table 1: Retargeting stressors at 100 Hz (V3). With the guard ON no capsule pair ever interpenetrates (min gap ≥ the 2.5 cm margin) and there are zero joint-limit violations; with it OFF the crossing arms pass *through* each other (−5.7 cm; the MJCF generates no arm–arm contacts) and the chest poke penetrates and jams 307 steps of joint-limit violation. The base holds still (<0.1 mm) when the operator's head is still. The guard's tracking cost is zero at rest (all pairs >11 cm > the 10 cm influence), bounded on a near-head pass (+9 mm mean on overhead), and large only when the command itself is unsafe (by design — the wall, not the operator, wins).

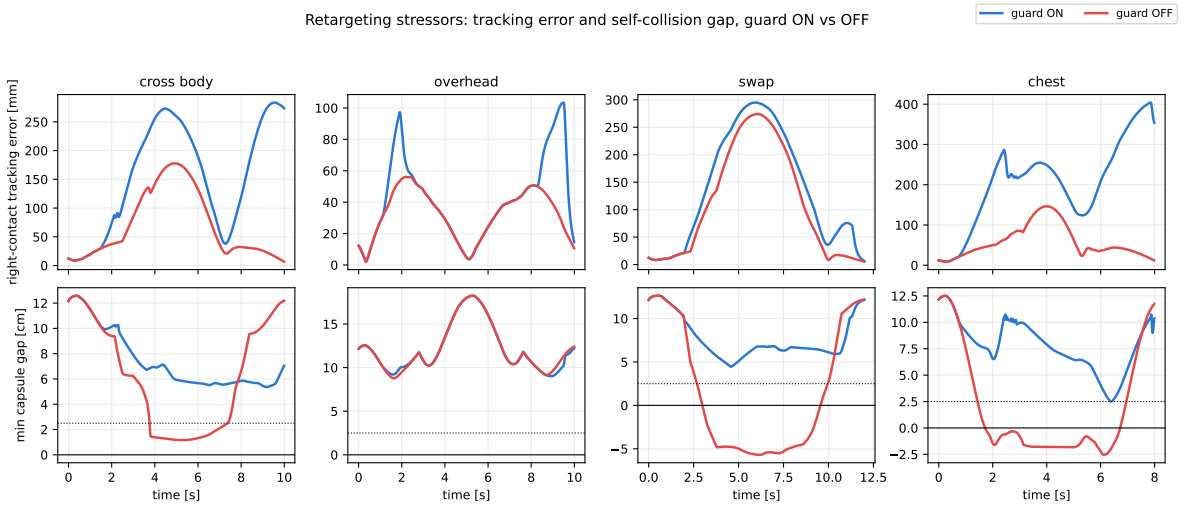


Figure 3: V3 traces: right-contact tracking error (top) and minimum capsule gap (bottom) per stressor, guard ON (blue) vs OFF (red). Dotted line = the 2.5 cm safety margin.

the fixed D makes the response *more* underdamped as K grows ($\zeta \propto 1/\sqrt{K}$); steady-state error is $< 0.35°$ for every K because the bias feed-forward cancels gravity exactly in simulation. A 40 N push deflects the palm 30 cm and it returns to within 3.6 mm on release; in compliance (K=0) the same class of push yields 86 cm and *holds* where left (<0.1 mm post-release drift — the documented yield-and-hold design, not a spring-back). The base stays put in `region=arm` (<0.1 mm during the push). The torque-domain soft wall stops a 30 N drag into the torso at a +1.2 cm capsule gap where the unguarded drag penetrates to −7.9 cm.

13.2 Retargeting latency: where the operator's lag actually lives

An operator reported (2026-07-28) that whole-body retargeting felt "way too slow . . . I need it to be fast and more real time." "Slow" is at least two different defects with different fixes — the robot starting *late* (dead time) versus starting on time and *crawling* (a rate limit) — so the first job was separating them rather than turning a knob.

Two harnesses do it. `server/tools/exp_retarget_latency.py` measures the whole loop over

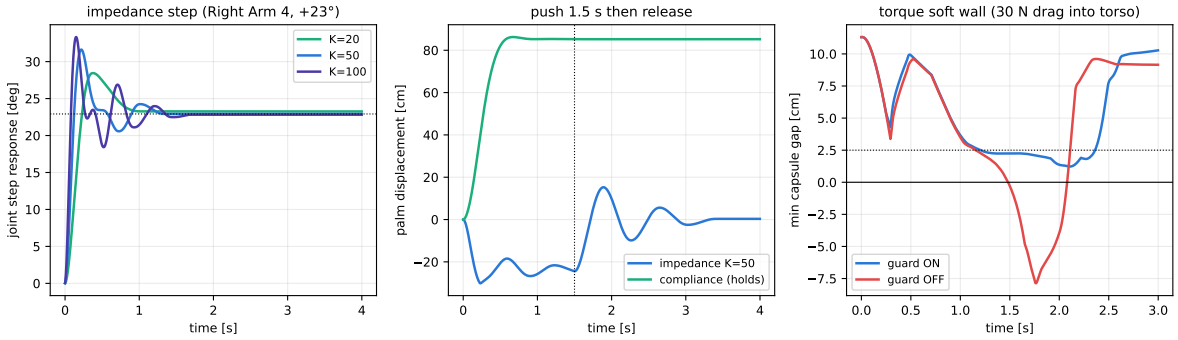


Figure 4: V4: impedance step responses vs stiffness (left), impedance push-recovery vs compliance yield-and-hold (middle), and the torque-domain self-collision soft wall (right).

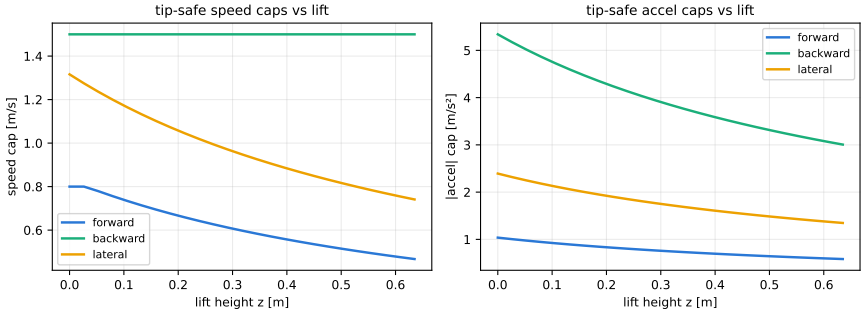


Figure 5: V1: tip-safe speed and acceleration caps vs lift height — monotone non-increasing per axis (forward binds; lateral/rear run nearer the tip limit for agility).

the live wire: it streams a scripted operator hand at the app's 30 Hz, forward-kinematics the joint angles that come back, and reports the step response to a 25 cm hand jump plus the cross-correlation delay under a continuous 0.4 Hz sweep — the second being what teleoperation actually is. `controller/experiments/exp_retarget_latency.py` then runs the same operator against the engine in-process with two different plants: an *ideal* one where every joint lands on its command instantly, and the real `mj_step` physics. The difference between the two is the servo's contribution, and it is not measurable from the wire alone.

The decomposition was unambiguous. Against the ideal plant the engine's following lag is 0 ms: the arm IK is a per-frame solve with no integrator, and neither `ARM_W_SMOOTH` nor the trust region binds at conversational hand speeds. Against real physics the same engine lagged 170 ms. *The lag was the arm servo, not the retargeter.*

The servo gains were then found to be inconsistent within the repository: `models/mabel_full.xml` ran the arm joints at $k_p = 100$, $k_v = 20$, while `models/right_arm.xml` has always run *the same joints with the same force limits* at $k_p = 500$, $k_v = 45$. Adopting the latter in the full model (with $k_v \propto \sqrt{k_p}$, which holds the damping ratio — raising k_p alone rings) cut the servo's contribution from 170 ms to 55 ms with no actuator-force saturation and only a mild rise in high-frequency content (a normalised ringing index of 0.93 to 1.21). A 9× cell reached 30 ms but at 1.77, and was not taken. Force ranges are the hardware limit and are unchanged.

That fixed continuous tracking but left the 25 cm step at 455 ms to 63 % of travel, with the arm's peak joint speed measured at 2.62 rad s^{-1} — exactly `ARM_TRUST_STEP` × f for the app's $f = 30$ Hz. The trust region was defined *per solve*, and its own comment justified the value as "0.10 rad @

60 Hz frames = a 6 rad s^{-1} per-joint slew — far above human teleop speed." The Vision Pro app later dropped to 30 Hz to kill a SwiftUI invalidation storm, and the slew silently halved to 3 rad s^{-1}, which is *not* above human teleop speed. Redefining it as a rate (`ARM_TRUST_RATE`, 6 rad s^{-1}, times the measured frame interval, capped by `ARM_TRUST_MAX`) restores the guarantee the constant always claimed, at whatever rate a client happens to stream.

	before	after	budget
step dead time [ms]	138	**79**	120
step t_{63} [ms]	467	**264**	350
step t_{90} [ms]	761	**449**	—
following lag [ms]	245	**140**	180
sweep amplitude retained	0.95	**1.00**	0.55

Table 2: Whole-body retargeting latency, measured over `ws://...:9090/teleop` against `server/run.sh`, before and after the arm-servo gains and the rate-defined trust region. Five of five checks pass after; two of five before.

A note on what did *not* change: the arm-fidelity battery (`server/tools/exp_arm_fidelity.py`, 9/9) and the tracking-channel battery (`exp_tracking_channels.py`, 16/16) both still pass, and every arm joint that can hold its parked target got *tighter* with the stiffer servo (±11° to ∼3°). Two joints cannot hold it at either gain: the wrist roll (±7 N m) sits saturated at exactly 7.00 N m, 45.8° out at the old gains and 57.1° at the new, because it must roll the 1.74 kg ORCA hand back against gravity; the elbow (±27 N m) saturates from some parking poses for the same reason. Both were verified in-process with the engine's gravity feed-forward running, and both predate the gain change — they are the same fact the OpenArm bring-up recorded, that this arm cannot hold itself on PD alone. The channel-park check therefore asserts on the joints that *can* reach their target and reports these two with their saturation, since asserting a target the actuator cannot hold tests nothing.

14 The configuration registry: `control_param.json`

Every tunable in the honesty list below (§15) is now catalogued in one human-editable file, `controller/control_param.json`, read through `controller/mabel/control_params.py`. The file began (2026-07) as the single source of the *arm* PD gains and gravity feed-forward — the numbers the setup studio's `armctl`, the real-robot arm law (`arm_control/gravity.py`), and `mabel_hw`'s per-joint command defaults all read, so they can never silently disagree. It was then expanded (2026-07-17) into the master registry for the *whole* control stack: the pure-PD position controller, WBC gravity compensation, the self-collision control barrier, the admittance base, the compliance workspace box, the whole-body IK, the base/tip-over motion model, the neck, the hands, and the lift/torso/body.

Gains and system identity live together. The arm block carries not only the per-joint $k_p/k_d/v_{\max}$ and the gravity-FF envelope (`enabled`, `clamp`, `scale`, `ramp_s`, `pose_pad`, hand-mass choice) but also the joint *system identity* — the firmware joint IDs (`l_j1...r_j7`), their MJCF joint-name map, the backend/link, the calibration slice, and the per-side torque/position/velocity ceilings. The calibration `dir` and raw window are deliberately *not* copied here: they are measured by the setup studio and owned by the `CalibrationStore` (the `calib_slice`), so this file stays the naming-and-limit registry without becoming a second, drifting copy of the calibration.

Block	Authoritative source today	Status
arm.gains / gravity_feedforward	control_param.json	LIVE
arm.impedance / rate_limit	arm_control/{impedance,rate_limit}.py	MIRROR
position_control	position_controller/pd.py (servo-inherited)	MIRROR
gravity_compensation	gravity_compensation/gravity_comp.py	MIRROR
collision_avoidance	config.py (SC_*)	MIRROR
admittance	config.py (ADMIT_*) + wrench_estimation.py	MIRROR
compliance	config.py (DRAG_*, LAZY_*)	MIRROR
whole_body_ik	config.py (IK_*)	MIRROR
base / tip_over	motion_model/tip_over.py, config.py	MIRROR
neck	config.py (NECK_*)	MIRROR
hands	hand_retargeting/*, config.py	MIRROR
lift / torso	config.py (LAZY_*), tip_over.py	MIRROR
*.hardware, body	mabel_hw/config.py	PLACEHOLDER

Table 3: The control_param.json registry: each block, where its numbers are authoritative today, and whether the JSON copy is live-wired, a documentation mirror, or a not-yet-configured placeholder.

Three honesty tags. Because most subsystems still read their numbers from controller/mabel/config.py (and the base/tip-over constants from motion_model/tip_over.py), each block carries a _source tag so the file never lies about what is wired:

- **LIVE** — read by running code today; editing the number changes robot behaviour with no code change. Currently the arm block (gains + gravity FF + rate limit).
- **MIRROR** — a documentation copy of a value that is *authoritative in config.py / tip_over.py*. Editing here does not yet change behaviour; the _authoritative field names the real source. Kept so this file is the one place to *read* every tunable. Migrating each MIRROR block to LIVE (having config.py load it from the JSON, exactly as the arm gains already do) is the tracked next step.
- **PLACEHOLDER** — the subsystem is not yet fully brought up on the real robot (hands' realized servo gains, the DaMiao body/neck/torso bus, the lift Pico); the numbers are conservative starting points or hardware defaults.

Table 3 maps each block to its authoritative source and status.

15 Every hardcoded part, in one place (honesty section)

Every constant below is now catalogued in control_param.json (§14); the values here and there are kept in sync.

- **Frame map** A = AVP_TO_ROBOT (fixed permutation); wrist-align identity.
- **Reach scale** default 1.2 (auto-calibrated, clipped $[0.5, 3.0]$).
- **Hand** per-finger SCALES (1.0); weights 20/150/0.5/0.02; finger/ joint/body name maps; fingertip pad offset 22 mm.
- **Arm** $W_{\text{wrist/elbow/orient/smooth}} = 600/30/20/2$; ARM_REG_W = $[0.08, 0.08, 0.40, 0.08, 0.02, 0.08, 0.60]$; wrist reg 0.03; HAND_REACH= 0.14 m; Arm7 yaw clamp ± 0.45 rad; ARM_MAXITER= 35; j7_lock_val (rest); wrist-flex gain/sign 1/1.

- **Neck** gain 1, per-axis signs $(+,-,+)$, local forward/up axes.
- **Whole-body** box $b_{\text{fwd/back/lat}} = 0.18/0.02/0.12$ m; weights $w_{\text{fwd/back/lat}} = 1.0/1.3/1.0$; $K_p = 3.5$, $K_{p,\text{yaw}} = 2$, $v_{\max} = 0.8$, $\omega_{\max} = 1.5$; torso gain 2.5; lift rate 0.25; front-line clamp ARM_WRIST_FRONT $= -0.05$ m.
- **Swerve** steering-error deadband 0.52 rad; module geometry from the XML.
- **Motion model** tip levers, CoM fit, safety factors, stop times, speed floors/ceilings — *measured from the chassis support polygon*; re-measure on any mass/geometry change.
- **Self-collision** capsule radii, monitored pairs, influence/margin/α (0.10/0.025/8); QP velocity cap 4 rad/s.
- **Safety** timeouts 0.4/0.5 s; retreat/engage 4/2.5 s; HOME lift 0; clutch pinch 0.03/0.055 m, double-window 0.6 s, cooldown 0.8 s.

16 Limitations

(1) **Two implementations** (§1) still coexist; the consolidation onto `mabel` is in progress and the cosine vs. DLS arm-IK choice is not yet unified. (2) **Walk/reach composition:** (1) scales the operator's global walk by s while the base follows 1:1, so during locomotion the arm target outruns the base by $(s-1)\times$ walk plus the box (≈ 200 mm); the front-line clamp masks the worst case but a cleaner fix scales only the hand-relative component. (3) **Reverse at full lift** is handled by a guard, not a fundamentally more stable base. (4) The **tip-over constants are geometry-specific** and must be re-measured if the chassis/payload changes; the envelope also ignores the *measured* payload / external wrench (the estimator in `force_estimation.py` exists but is not fed into `tip_usage` — a shrink-only coupling is the recommended next step, cf. ZMP disturbance observers). (5) **No force/haptic feedback** — the loops close vision and motion, not touch. (6) The self-collision projector slows both arms uniformly through the QP coupling and guards only distal links vs. torso/head/opposite arm; since the MJCF generates no arm–arm contacts, it is the *sole* defense for that pair (§10). (7) The torque engine's **joint damping is a fixed** D=0.6, so impedance overshoot grows with the stiffness slider ($24 \to 45\,\%$ for K $20 \to 100$, §13.1); a critical-damping schedule $D \propto \sqrt{K}$ would be the minimal fix. (8) Under **combined-twist target switches** the instantaneous ZMP usage can transiently exceed 1 (bounded by the ≥ 1.2 safety factors; physically verified upright, §9).

References

[1] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, 2004.

[2] E. Papadopoulos, D. Rey, "A new measure of tipover stability margin for mobile manipulators," *IEEE ICRA*, 1996.

[3] O. Khatib, "A unified approach for motion and force control," *IEEE J. Robotics and Automation*, 1987.

[4] A. D. Ames *et al.*, "Control barrier functions: theory and applications," *ECC*, 2019.

[5] S. Buss, "Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods," 2004.